

SIMATS ENGINEERING ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Ramamoorthy

QUESTIONS: 6

Total Marks:30

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I (K. Deepak / 192320078) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Short Answer Test

1. Smart Banking System

(a) Explain the following Object-Oriented Concepts

Class

A class is a blueprint or template that defines the properties (attributes) and behaviors (methods) of objects.

Example: BankAccount class contains account number, balance, deposit(), and withdraw() methods.

Object

An object is an instance of a class that has its own data and can perform actions.

Example: John's savings account is an object of the BankAccount class.

Message Passing

Message passing is the communication between objects through method calls.

Example: A Customer object sends a message to the BankAccount object by calling transferFunds().

Encapsulation

Encapsulation is the process of hiding data and allowing access only through public methods.

Example: The account balance is private and can only be modified using deposit() or withdraw() methods.

Polymorphism

Polymorphism allows the same method to perform different actions depending on the object.

Example: Different account types (Savings and Current) implement the calculateInterest() method differently.

(b) Explain the Object Relationships

Association

A general relationship where two objects interact independently.

Example: A customer uses a bank account.

Aggregation

A weak "has-a" relationship where objects can exist independently.

Example: A bank has many customers, but customers can exist even if the bank

closes.

Composition

A strong "part-of" relationship where the child object cannot exist without the parent.

Example: A bank account contains transaction records that are deleted if the account is deleted.

(c) Importance

- Promotes code reusability.
- Improves maintainability.
- Reduces duplication.
- Makes software easier to extend and modify.
- Enhances security through encapsulation.

2. Smart Healthcare Management System

(a) Object-Oriented Design Principles

Abstraction

Shows only essential features while hiding implementation details.

Encapsulation

Protects data by restricting direct access.

Modularity

Divides software into independent modules for easier development and maintenance.

(b) SOLID Principles

Liskov Substitution Principle (LSP)

A subclass should be able to replace its parent class without affecting program correctness.

Example: A Doctor class can replace an Employee class wherever required.

Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

Example: Separate interfaces for Appointment Management and Billing instead of one large interface.

(c) Benefits

- Improves flexibility.
- Reduces bugs.
- Makes maintenance easier.
- Supports reusable code.
- Simplifies future enhancements.

3. Online Food Delivery System

(a) SDLC Phases

1. Planning

Determines project objectives, feasibility, budget, and schedule.

2. Requirement Analysis

Collects and documents user and system requirements.

3. Design

Creates system architecture, database, UI, and software design.

4. Development

Developers write the source code according to the design.

5. Testing

Tests the software to identify and fix defects.

6. Maintenance

Provides updates, bug fixes, and performance improvements after deployment.

(b) Purpose of Each Phase

Phase	Purpose
Planning	Define project scope and feasibility
Requirement Analysis	Understand customer needs

Phase	Purpose
Design	Prepare software architecture
Development	Build the application
Testing	Ensure software quality
Maintenance	Improve and support software after release

(c) How SDLC Delivers Reliable Software

- Provides a systematic development process.
- Detects errors early.
- Improves software quality.
- Reduces project risks.
- Ensures customer requirements are met.

4. Smart Agriculture Monitoring System

(a) SDLC Models

Prototype Model

Develops a working prototype to gather user feedback before the final product.

Spiral Model

Combines iterative development with continuous risk analysis.

Agile Model

Develops software in short iterations with continuous customer feedback.

DevOps Model

Integrates development and operations for continuous integration, testing, and deployment.

(b) Prototype Model vs Agile Model

Prototype Model

Builds a prototype first

Agile Model

Builds working software in iterations

Prototype Model

Focuses on understanding requirements

Customer feedback mainly on prototype

Less flexible after final design

Agile Model

Focuses on continuous delivery

Customer feedback throughout development

Highly flexible to requirement changes

(c) Recommended Model – Agile Model

Justification

- Requirements change frequently.
- Allows continuous customer feedback.
- Delivers software in small increments.
- Easy to modify features.
- Faster delivery and improved customer satisfaction.

5. Smart Transport Management System

(a) Importance of UML

UML (Unified Modeling Language) is a standard visual language used to design and document object-oriented systems.

Importance

- Visualizes system design.
- Improves communication.
- Helps developers understand requirements.
- Simplifies maintenance.
- Supports documentation.

(b) UML Diagrams

State Diagram

Shows different states of an object and transitions between them.

Component Diagram

Represents software components and their relationships.

Deployment Diagram

Shows the physical deployment of hardware and software components.

Package Diagram

Groups related classes into packages to organize large systems.

(c) Benefits of UML

- Improves stakeholder communication.
- Reduces misunderstandings.
- Provides clear documentation.
- Simplifies system maintenance.
- Supports better software design.

6. Smart Inventory Management System

(a) Modular Software Design

Modular software design divides a system into independent modules, each performing a specific function.

Example Modules

- Inventory Management
- Supplier Management
- Sales
- Reporting

(b) Procedural Programming vs Object-Oriented Programming

Procedural Programming

Focuses on functions

Data and functions are separate

Less reusable

Difficult to maintain

Object-Oriented Programming

Focuses on objects

Data and methods are combined

Highly reusable

Easier to maintain

Procedural Programming

Object-Oriented Programming

Limited security

Better security through
encapsulation

(c) Benefits of Modularity

Scalability

New modules can be added without affecting existing modules.

Maintainability

Individual modules can be modified independently.

Reusability

Modules can be reused in other projects, reducing development time and cost.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps